Manual de Pruebas - BTG Funds API + Frontend React

Fecha: 2026-04-28 12:57

1. Objetivo

Este manual describe como validar el sistema completo (frontend + backend) en ambientes Local y AWS, incluyendo autenticacion, suscripcion, cancelacion, historial y validacion de logs.

2. URLs Oficiales del Proyecto

- Frontend AWS (CloudFront): https://d2jbrhwnav2xw1.cloudfront.net
- Backend AWS (Elastic Beanstalk): http://pt-be-amaris.us-east-1.elasticbeanstalk.com
- Swagger AWS UI: http://pt-be-amaris.us-east-1.elasticbeanstalk.com/swagger-ui.html
- Swagger AWS OpenAPI: http://pt-be-amaris.us-east-1.elasticbeanstalk.com/v3/api-docs
- Frontend local: http://localhost:5173
- Backend local: http://localhost:8081
- Swagger local UI: http://localhost:8081/swagger-ui.html

3. Credenciales de Prueba

Usuario semilla:

- Documento: 123456789
- Password: ChangeMe123!

4. Pruebas desde Frontend (React)

1. Abrir https://d2jbrhwnav2xw1.cloudfront.net
2. Iniciar sesion con las credenciales de prueba.
3. Cargar fondos y validar que se liste al menos un fondo.
4. Seleccionar canal de notificacion (EMAIL o SMS) y suscribirse.
5. Copiar el subscriptionId generado en la confirmacion.
6. Consultar historial y validar transaccion de suscripcion.
7. Cancelar suscripcion con el subscriptionId y canal deseado.
8. Consultar historial nuevamente y validar transaccion de cancelacion.

5. Pruebas desde Postman

Importar los archivos incluidos en /postman:

- funds-api.postman_collection.json
- funds-api.local.postman_environment.json
- funds-api.aws.postman_environment.json

Ejecutar primero "01 - Login" y luego continuar secuencialmente. La coleccion guarda automaticamente token y subscriptionId.

6. Checklist de Aceptacion

- Login responde 200 con JWT.
- GET /funds responde 200 con catalogo de fondos.
- POST /subscriptions responde 201 y retorna id de suscripcion.
- DELETE /subscriptions/{id} responde 200 con mensaje de salida.
- GET /transactions muestra eventos de suscripcion y cancelacion.
- Logs incluyen requestId, status, duracion y evento de notificacion mock (EMAIL/SMS).

7. Troubleshooting

- Pantalla blanca frontend: validar VITE_API_BASE_URL y ejecutar invalidacion CloudFront (/*).
- 502 en EB: validar que backend escuche en puerto 5000 (SERVER_PORT=5000).
- Error de DB: validar DB_URL con prefijo jdbc:postgresql:// y credenciales RDS.
- 401 en endpoints protegidos: repetir login y enviar Authorization Bearer token.
- CORS: validar origen frontend en WebConfig del backend.

8. Variables de Entorno Recomendadas

- Backend EB: SPRING_PROFILES_ACTIVE, DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET, SERVER_PORT=5000
- Frontend build: VITE_API_BASE_URL

Anexo A - Guia de Pruebas End-to-End (Local y AWS)

Fecha de actualizacion: 2026-04-28 14:05

1. Enlaces y Paths del Proyecto

- Backend local (Swagger): http://localhost:8081/swagger-ui.html
- Backend AWS EB (Swagger): http://pt-be-amaris.us-east-1.elasticbeanstalk.com/swagger-ui.html
- Frontend local (Vite): http://localhost:5173
- Frontend AWS (CloudFront): https://d2jbrhwnav2xw1.cloudfront.net
- Coleccion Postman: /postman/funds-api.postman_collection.json
- Environment local Postman: /postman/funds-api.local.postman_environment.json
- Environment AWS Postman: /postman/funds-api.aws.postman_environment.json

2. Flujo de Pruebas Local (Paso a Paso)

9. Levantar backend con perfil local y validar Swagger en /swagger-ui.html.
10. Levantar frontend en carpeta /frontend con npm run dev y abrir http://localhost:5173.
11. En Postman, seleccionar environment local y ejecutar en orden: login -> funds -> subscribe -> transactions -> cancel -> transactions.
12. Verificar logs en backend: AuthService, SubscriptionService y MockNotificationAdapter/SnsNotificationAdapter segun provider.

3. Flujo de Pruebas AWS (Paso a Paso)

13. Validar estado de Elastic Beanstalk en Health: Green/Ok y version desplegada activa.
14. Abrir Swagger AWS: http://pt-be-amaris.us-east-1.elasticbeanstalk.com/swagger-ui.html.
15. Abrir frontend productivo en CloudFront: https://d2jbrhwnav2xw1.cloudfront.net.
16. Ejecutar coleccion Postman con environment AWS en el orden funcional definido.
17. Corroborar en logs de EB: requestId, status HTTP, tiempos de respuesta y eventos de notificacion.

4. Matriz de Endpoints, Request y Datos de Prueba

Orden	Metodo y Path	Auth	Request de ejemplo	Resultado esperado
1	POST /api/v1/auth/login	No	{"documentNumber":"123456789","password":"123456"}	200 + token JWT
2	GET /api/v1/funds	Bearer token	Sin body	200 + lista de fondos
3	POST /api/v1/subscriptions	Bearer tok	{"fundId":1,"notificationChannel":"EMAIL","email":"correo@dominio.com"}	201 + subscriptionId +

		en		notificacion
4	GET /api/v1/transactions	Bearer token	Sin body	200 + historial de transacciones
5	DELETE /api/v1/subscriptions/{id}	Bearer token	Query: notificationChannel=EMAIL&email=correo@dominio.com	200 + mensaje de cancelacion + devolucion saldo
6	POST /api/v1/subscriptions (SMS)	Bearer token	{"fundId":2,"notificationChannel":"SMS","phoneNumber":"+573001112233"}	201 + notificacion SMS (SNS o MOCK)

5. Variables de Entorno Recomendadas

- SPRING_PROFILES_ACTIVE=aws
- SERVER_PORT=5000 (requerido en Elastic Beanstalk para proxy por defecto)
- DB_URL=jdbc:postgresql://funds-db-rds2.cgxoc8kks46u.us-east-1.rds.amazonaws.com:5432/postgres?sslmode=require
- DB_USERNAME=postgres
- DB_PASSWORD=<tu_password>
- JWT_SECRET=<secreto_32+_caracteres>
- NOTIFICATION_PROVIDER=SNS (o MOCK para pruebas sin AWS SMS)
- AWS_REGION=us-east-1
- SNS_EMAIL_TOPIC_ARN=arn:aws:sns:us-east-1:746902954826:funds-email-notifications
- VITE_API_BASE_URL=http://pt-be-amaris.us-east-1.elasticbeanstalk.com

6. Decisiones Tecnicas y Justificacion

- Spring Boot 3 + Java 21: stack estable, moderna y lista para produccion con buen soporte en AWS.
- Arquitectura hexagonal (Ports & Adapters): separa dominio de infraestructura, mejora mantenibilidad y testabilidad.
- PostgreSQL (RDS): consistencia transaccional para operaciones de suscripcion/cancelacion y trazabilidad bancaria.
- JWT + roles: control de acceso stateless, ideal para microservicios y despliegues escalables.
- Swagger/OpenAPI: contrato vivo para QA, frontend y consumidores externos.
- Logback estructurado JSON: observabilidad, trazabilidad por requestId y analisis centralizado en CloudWatch.

- SNS para notificaciones: servicio administrado, desacoplado y escalable para eventos de negocio.
- S3 + CloudFront para frontend: hosting estatico de bajo costo, alto rendimiento global y despliegue simple.
- Elastic Beanstalk para backend: acelera despliegue de JAR, configuracion por variables de entorno y operacion gestionada.

7. Troubleshooting Rapido

- Error DB_URL: verificar prefijo correcto jdbc:postgresql:// (sin errores de formato).
- Error 502 en EB: asegurar SERVER_PORT=5000 para que nginx enrute al proceso Java.
- No llega EMAIL SNS: confirmar subscription de correo en estado Confirmed.
- No llega SMS SNS: validar permisos IAM, gasto mensual SMS y estado sandbox/habilitacion de servicio.

Fin del anexo.

Cierre de Entrega

Este documento consolida el estado final del proyecto BTG Funds API + Frontend, incluyendo arquitectura, despliegue, pruebas funcionales y evidencia de operacion en entornos Local y AWS.

Resumen Ejecutivo

- Backend desplegado en Elastic Beanstalk con perfil aws y conexion activa a RDS PostgreSQL.
- Frontend desplegado en S3 + CloudFront con comportamiento SPA y enrutamiento de /api/* hacia backend.
- Autenticacion JWT, gestion de fondos, suscripciones, cancelaciones y consulta de transacciones operativas.
- Notificaciones por EMAIL integradas con SNS (confirmadas); SMS documentado con ruta de habilitacion pendiente por cuenta AWS.

Checklist de Entrega

- [OK] Codigo fuente backend y frontend versionado y compilable.
- [OK] Swagger disponible y accesible en entorno AWS.
- [OK] Coleccion Postman y environments (Local/AWS) actualizados.
- [OK] Manual de pruebas funcionales con pasos reproducibles.
- [OK] Logs estructurados para trazabilidad de requests y operaciones.
- [OK] Infraestructura desplegada y validada (EB, RDS, S3, CloudFront, SNS EMAIL).

Evidencias Recomendadas para Presentacion

- Login exitoso y obtencion de JWT.
- Consulta de fondos.
- Suscripcion con notificacion EMAIL recibida.
- Cancelacion de suscripcion con devolucion de saldo.
- Consulta de historial de transacciones.
- Pantallazo de CloudFront funcional y Swagger en AWS.
- Fragmentos de logs de backend con requestId y status HTTP.

Riesgos Conocidos y Siguientes Pasos

- Habilitar SMS productivo en AWS End User Messaging / Pinpoint SMS Voice V2 para reemplazar pruebas mock.
- Aplicar dominio personalizado y TLS administrado para frontend/backend.
- Agregar pipeline CI/CD (build, tests, deploy) para reducir despliegues manuales.
- Fortalecer observabilidad con dashboards y alarmas CloudWatch por endpoint critico.

Estado Final

Estado: ENTREGA COMPLETA Y FUNCIONAL

Fecha de cierre: 2026-04-28 15:01

Clonacion del Repositorio

Repositorio oficial:

- https://github.com/JNASAR24/Prueba_T-cnica-Desarrollo_BackEnd-Amaris.git

Paso a paso:

18. Abrir terminal (PowerShell, CMD o Git Bash).
19. Ubicarse en la carpeta destino.
20. Ejecutar: git clone https://github.com/JNASAR24/Prueba_T-cnica-Desarrollo_BackEnd-Amaris.git
21. Entrar al proyecto: cd Prueba_T-cnica-Desarrollo_BackEnd-Amaris
22. Validar ramas: git branch -a
23. (Opcional) Actualizar: git pull origin master

Requisitos previos:

- Git
- Java 21
- Maven 3.9+
- Node.js 18+

Seccion actualizada: 2026-04-28 15:12